

Usable software at the end of each takt

A milestone in the lean transformation of a globally distributed software development team

Roopa M. S., C. Sankarasubbiah
Siemens Technology and Services Pvt. Ltd.
Bangalore, India
roopa.ms@siemens.com, c.sankarasubbiah@siemens.com

V. S. Mani
Siemens Healthcare Pvt. Ltd.
Bangalore, India
vs.mani@siemens.com

Abstract— This experience paper presents how a globally distributed software engineering team was able to deliver usable software at the end of each takt, why this was important, and the benefits derived. We also describe the approach taken, the challenges faced and the steps to overcome them..

Keywords—global software engineering; lean; usable software

I. INTRODUCTION

In this experience paper we present an important part of our journey towards lean and how we successfully reached the important milestone of delivering usable software at the end of takt.

A. What is a takt?

A takt is similar to a sprint. Whereas a sprint is a time-boxed iteration that is specific to software development projects, a takt is fixed duration of time that applies to all departments that are engaged in the development of a product.

B. Importance of takts

A takt thereby becomes the basis for synchronizing all product development activities, and ensures coordination across all departments involved in developing the product. Synchronization is important as uncoordinated activities can potentially result in imbalances between capacity and available resources. In addition, uncoordinated handoffs can result in inefficiencies characterized by bottlenecks, and quality problems.

II. BACKGROUND

Our organization is part of a global team that develops products in a highly regulated industry. We are involved in developing hardware, software, and firmware for multiple product families. The products must fulfill regulatory requirements that entail conforming to industry standards across all versions of the product encompassing different versions of firmware or third-party components. Given the mission-critical nature of our products, quality is non-negotiable.

The team had been operating for over 10 years. They had a defined framework to address the issues of communication, coordination, and control that are encountered in global software development [1]. The team also had an approach for effective people management [2].

The team switched to lean software development methodology to ensure on-time delivery of high-quality software. The drivers for transitioning to lean were: i) shortening the time between releases, ii) reduce the time spent in planning, iii) challenges in integrating sub-systems [3].

As the project involved developing devices with hardware, firmware and software, there was a strong motivation to ensure that usable software was available at the end of each takt. It also was in line with the deliver as fast as possible principle of lean [4].

III. INITIAL IMPLEMENTATION OF LEAN

A. Project organization

The project team was distributed over four locations in three countries. The project team was split into multiple takt teams. While the takt teams were in different locations, members of a takt team were collocated. The teams used a common set of development tools across locations.

B. Takt team composition

Each takt team typically had about seven to ten team members. A takt team brings in all relevant roles into a single self-sufficient team and hence consisted of cross-functional team members who were responsible for realizing the requirements [5]. The good way to understand a takt team is to consider it as the small company within the organization that owns a set of requirements.

A takt team is similar to a scrum team in that it contains cross functional competences. In a pure software only product development, it is the same as a scrum team.

The representative of the takt team was the takt speaker, who was usually an individual with deeper knowledge of the product and domain. The role of the takt speaker is similar to that of a scrum master in agile. The takt speaker is responsible for defining the flow of requirement realization activities, the interdependencies among takts and different projects, and other coordination tasks.

The takt teams also had a few dedicated test experts who were responsible for defining the test cases for the requirements realized in a takt and for executing these test cases. The tests included regression testing, integration testing, testing of non-functional requirements and specific tests required to conform to regulations.

C. Nature of requirements

Due to the high level of interdependence among the components, a single requirement spanned multiple takt teams that were based in different locations. Though we adopted a takt-based approach for the implementation and testing activities, we continued managing requirements in a traditional way, since:

- There was a high level of dependency between the different software and hardware components, and in addition the third-party modules.
- There were a large number of requirements, and most of them were big and complex.

D. Addressing the requirements

From the total set of requirements, a subset was selected for each takt, and from this subset specific requirements were assigned to individuals by the takt speaker.

To ensure the required test and delivery objectives of the product were fulfilled, we defined separate hardening takts dedicated to fixing defects and testing of the integrated product in the entire system. No new requirements were realized in the hardening takts.

This approach did significantly help in increasing the speed and predictability in delivering software. However, we faced issues that hindered our ability to deliver usable software at the end of each takt.

IV. ISSUES FACED IN INITIAL IMPLEMENTATION OF LEAN

Despite re-engineering the processes [3] we still faced issues, and the main objective to have usable software after every takt was not achieved. In addition, several requirements were still spanning multiple takts.

A. Development teams being spread across multiple locations globally

In many cases, realizing a requirement needed the participation takt teams based in different locations and in different time zones. As a result, seeking and getting clarifications took more time, which made it difficult to meet the takt deadlines.

In addition, we observed that team members were hesitant in openly voicing their opinions with team members in different locations.

B. Requirements prioritized over tests

With requirements being prioritized over tests, more new requirements were being included in a takt. This in turn required creating and executing an even larger number of tests within the takt. Since more time was taken towards implementing requirements, the time available to complete tests was inadequate. Increasingly, the requirements developed in a takt were not tested sufficiently, and increasingly, a number of tests were being deferred to subsequent takts and in some cases even to the hardening takts. As a result, the defects in the newly realized requirements were not being detected in the same takt. In addition, due to the priority to realize requirements, the defects detected in a takt were not fixed in

the same takt. On several occasions, despite requirements being realized, the takt deliverables were not completed, since these could not be tested.

Assigning a higher priority to requirement realization also resulted in skewing the test effort across the takts. There was a much higher test load in the later takts, which resulted in undue pressure on the test experts to complete the testing in less time, which was counter-productive.

Issues related to non-functional requirements were being uncovered mainly in the hardening takts. Due to the difficulty in reproducing the defects related to non-functional requirements, identifying the root causes and solutions is typically time and effort intensive. The shortage of time for testing made it difficult to detect and resolve such issues effectively. Even after allotting the hardening takts for testing before release, on several occasions we still observed field issues in the customer environment.

Defects detected in previous releases/versions of the product (legacy defects) were assigned a lower priority compared to new requirements. This resulted in legacy defects getting accumulated and these were increasingly being fixed in the hardening takts. The fixing of many defects in a short time towards the end of the project impacted quality. Since quality was could not be compromised, it introduced delays in delivery.

C. Delayed integration

Since it needed multiple takts to complete development of a complex requirement, it was only possible to integrate the associated components in the later takts. In some cases, the integration was only possible in the final development takt. As a result, integration-related issues were identified towards end of the development takts.

Fixing these issues needed considerable effort from all takt teams and resulted in a substantial increase in the work load towards the end of the project. It was difficult to address these issues effectively in the limited time available, which in turn impacted quality and delivery time. The late integration of the code developed by the independent takt teams also resulted in collaboration-related issues. It was not uncommon to see the different takt teams pointing fingers at each other with regard a specific issue, leading to a waste of time and reducing trust.

D. Increased dependency on a few key resources

There was an increasing dependency on a few key team members, due to their specialized knowledge which spanned across multiple components. More requirements were being assigned to these key team members as it was difficult to distribute load among team members, which resulted in new bottlenecks.

E. Culture and mindset of team

As requirements were assigned by the takt speaker to team members, they assumed the overall responsibility was with the takt speaker, and didn't own the task in spirit. In addition, the team members were more focused on achieving individual goals by completing tasks assigned to them, rather than focusing on achieving the takt team goals and the overall project goals.

Perhaps most importantly, the team members were not convinced that in a single takt, they could meaningfully realize requirements, detect defects, and fix these defects.

V. ANALYZING THE ROOT CAUSES

At the end of every takt, we conduct a retrospective meeting where we discuss the issues faced, what went well, what went wrong, and what practices we should continue. Action items are defined to address the aspects that went wrong. In addition, at the end of each version, we have a lessons learned meeting where inputs from all the retrospective meetings are consolidated and the root causes for different issues are identified. The root causes we identified are listed below.

- Developing a single requirement involved takt teams in different locations.
- Large and complex requirements that could not be completed in a single takt.
- Prioritization of realizing requirements over testing and fixing defects (managing technical debt).
- Integration of components deferred to later takts.
- Non-functional requirements not getting adequate focus in the early stages of realizing a requirement.
- Insufficient build and test automation and lack of scalable infrastructure.
- Insufficient knowhow sharing resulting in increased dependency on a few key resources.
- Culture and mindset change to ensure usable software at the end of each takt.

After identifying the root causes, the takt teams proposed measures to address them. The teams then collectively evaluated the approaches for feasibility. Eventually, a few measures were applied. These are described in the next section.

VI. MEASURES TAKEN TO OVERCOME ISSUES AND THE RESULTS OF THE MEASURES

A. Minimizing cross location involvement in developing a single requirement

Knowledge transfer sessions were introduced to ensure that the necessary competences to develop a single requirement were available in a single location. Where collocation of required competences was not possible at a single location, teams in different locations participated in the development of a single requirement. To enable this, we institutionalized daily virtual standup meetings involving all takts in all regions. Fortunately this was possible due to favorable location distribution. We observed that the daily virtual stand up meetings were not effective in addressing the synchronization issues that were common in the globally distributed team. Addressing these issues involved detailed discussions and arriving at common approaches. Hence, we institutionalized separate synchronization meetings at predefined time slots that provided time to discuss and resolve such issues. It was

mandatory for all stakeholders to attend the synchronization meetings.

Results:

- Single requirement realized in single location wherever possible.
- No synchronization issues

B. Splitting large and complex requirements to smaller and simpler requirements

We applied the principles of user-centered design to split large and complex requirements into smaller requirements that could be completed within one takt. To do this we conducted workshops that facilitated a dialog among the relevant stakeholders (product owners, domain experts, architects, test experts, developers and usability experts). In the workshops the requirements were initially split into tasks, and subsequently into user stories [6, 7]. A task is an activity of a user which is closed and independent from other tasks. As the team is fully aware of the context in which the user performs the task, it could be translated into a user story. In this way, a single requirement was split into several user stories with fine-grained descriptions. Development of detailed user stories was completed prior to start of the takt and were sent for review across all locations to help minimize need for seeking and getting clarifications across locations which took more time.

Results:

- Better clarity of the requirements and better understanding of the requirements achieved due to fine-grained detailing.
- Progress was transparent and clear to all stakeholders. The closed (completed) and open (not started) requirements were distinct and work in progress was minimized.
- Able to deliver the entire version addressed with most important requirements on time and with specified quality.

C. Prioritizing requirements effectively

We demanded that user stories be defined at least two takts in advance. This meant that the user stories for the first and second takts were defined in the planning phase. As the user stories contained fine-grained details, we were able to identify the interdependencies readily. While planning a takt only user stories with no unresolved dependencies were included. To ensure a requirement was completed in the time of a single takt, wherever possible, the user stories originating from the same requirement were taken up together in separate takt teams working in parallel. The conversations in the periodic synchronization meetings went a long way in ensuring this.

Results:

- Delays due to dependencies were reduced.
- Most important requirements were prioritized and completed in the initial takts.

D. Enabling early integration of components

We sensitized the team about the importance of completing a user story before taking on the next user story. The mantra was “stop starting and start finishing.” In addition, we made it mandatory that the code satisfied the defined completion criteria, and among other things, passed all necessary unit tests, passed the planned automated tests (tests had to fulfill the defined coverage goals), and none of the known defects related to the realized requirement remain unfixed. We also strictly enforced the defined release criteria for each component, which specified the number of permitted open legacy defects for each severity category. This helped ensure that the teams not only focused on completing requirements but also committed effort to fixing legacy issues. Further, in the common configuration management system, we introduced a takt-specific branch and an integration branch, in addition to the main branch to avoid build issues among the globally distributed teams. These measures helped ensure that we could readily integrate the relevant changes or realized requirements and commence testing of the requirements realized in a takt. It also ensured that at every point in time the code on main branch was ready for system testing.

Results:

- Early detection of defects arising from integration of components.
- Sufficient time for analysis and proper fixing of integration-related defects.
- Wait-time to get a build for system testing reduced to zero.
- Number of legacy defects always in control due to strictly enforced completion criteria.

E. Ensuring focus on non-functional requirements across the entire duration of a takt

To ensure defects related to non-functional requirements were detected in the takt testing itself, we automated most of the stress and load monitoring test cases. These automated test cases could be executed from any location. All takt teams were mandated to execute these automated tests prior to merging a realized requirement into the takt branch. Typically the automated tests were executed daily, which helped in detecting and isolating non-functional issues such as memory leaks sooner, and provided the opportunity to quickly fix them.

Result:

- No delays due to non-functional issues.

F. Appropriate automation and scalable infrastructure

To ensure we were able to simulate typical customer environments, we set up multiple large configurations. This allowed the teams to execute the tests on these configurations from the first takt onwards [8, 9]. We automated the setup of different environments, which enabled developers to test, merge (with takt branch) and integrate (with integration branch). We optimized the automated test cases to avoid unnecessary wait times. We also adapted the test suites to enable parallel execution. Though the infrastructure was

accessible to all teams, some system tests still require proximity to specific equipment (e.g. when the equipment does not provide soft switches) and need to be run from a single location.

Results:

- Significant reduction in field defects.

G. Systematic knowledge management to avoid bottlenecks due to lack of available skills

Software development involves a significant amount of learning [5]. We mapped the components to the individuals in the project, who had a deep understanding of a component. The team member with the deepest knowledge of a component was made the primary component owner. In addition, we identified two other team members as secondary and tertiary component owners. We then defined training programs for the secondary and tertiary component owners. This ensured that across the entire project we had at least three team members who could independently take on development tasks related to a component. We also identified code experts and requirements experts, who in addition to their primary roles, were tasked with leading code reviews and circulating comments. The global team has a portal where technical information related to the components and the product is centrally available. Each team member is expected to contribute to the portal. We also introduced a periodic refresher course on Lean and requirements engineering.

Results:

- Avoided resource/skill bottlenecks.
- Collective expertise of team increased.
- Increased sharing of know-how and learning from each other.

H. Changing the culture and mindset

To overcome cultural barriers, such as the hesitation to openly voice opinions, we defined an escalation matrix, which specified who to contact for what type of issue. For example, for issues related to requirements, any team member could contact the product owners, and domain experts. In addition, any team member could ‘stop the line’ if they observed issues in quality.

We empowered the team members in different ways [10]. To begin with they could select the user stories they would work-on based on their expertise. This allowed them to proactively evaluate the effort, seek clarifications as needed, and assess the dependencies before committing – thereby enhancing the ownership mindset. We also set team goals and laid greater emphasis on team achievement over individual achievement. As part of this, team members were encouraged to take up activities outside their primary role. For instance, developers could work-on testing when the testers were fully loaded.

For a more complete set of interventions defined to change the culture and mindset of the team see [11].

Results:

- Team goals resulted in greater collaboration and in many cases developers willingly supported the other team members to complete their tasks.
- The issues raised through the escalation matrix were shared through the virtual stand up meetings, which went a long way in addressing them effectively.

VII. SUMMARY

The summary of the root causes, measures, and results is presented in table 1.

TABLE I. SUMMARY OF ROOT CAUSES, MEASURES, AND RESULTS

Sl. No.	Root cause	Measure	Result
1	Minimizing cross location involvement in developing a single requirement	Knowledge transfer sessions introduced.	Single requirement realized in single location wherever possible. No synchronization issues.
2	Large and complex requirements that could not be completed in a single takt.	Split large and complex requirements into smaller requirements that could be completed with-in one takt.	Delays due to dependencies were reduced. Most important requirements were prioritized and completed in the initial takts.
3	Prioritization of requirements over testing and fixing defects (managing technical debt).	User stories originating from the same requirement were taken up together in separate takt teams working in parallel. User stories defined at least two takts in advance.	Delays due to dependencies were reduced. Most important requirements were prioritized and completed in the initial takts.
4	Integration of components delayed to later takts.	<i>Stop starting and start finishing</i> mantra adopted. Made it mandatory that code satisfied the defined completion criteria. Enforced the defined release criteria for each component.	Early detection of defects. Sufficient time for analysis and proper fixing. Wait-time to get a testable build reduced to zero. Number of legacy defects always in control
5	Non-functional requirements not getting adequate focus.	Automated the stress and load monitoring test cases.	No delays due to non functional issues.
6	Insufficient automation and lack of scalable infrastructure.	Automated the setup of different environments. Optimized automated test cases to avoid unnecessary wait times adapted the test suites to enable parallel execution. Set up multiple large configurations.	Significant reduction in field defects.
7	Increased dependency on a few key resources	Systematic knowledge management to avoid bottlenecks due to lack of available skills. Identified and trained team members to be primary, secondary, and tertiary component owners.	Avoided resource/skill bottlenecks. Collective expertise of team increased Increased sharing of know-how and learning from each other.
8	Changing the culture and mindset	Empowered the team members to pick the requirements based on their expertise. Set team goals and laid greater emphasis on team achievement over individual achievement.	Team goals resulted in greater collaboration and in many cases developers willingly supported the other team members to complete their tasks.

VIII. CONCLUSION

The new approach helped the team achieve the goal of delivering usable software at the end of each takt. This was a significant milestone in our journey towards lean. The achievement was laudable and appreciated within the organization. As the approach demands maturity within the team, it takes time to realize the benefits. However, it is well worth the effort.

REFERENCES

- [1] TAPER: a generic framework for establishing an offshore development center, G. Hoefner, V. S. Mani, 2nd IEEE International Conference on Global Software Engineering, 2007
- [2] 4 C: An approach for effective people management in an offshore software development center, G. Hoefner, V. S. Mani, 7th IEEE International Conference on Global Software Engineering, 2012
- [3] Lean Transformation: How Lean helped to achieve Quality, Cost and Schedule, Uma V., IEEE 9th International Conference on Global Software Engineering, 2014
- [4] Lean Software Development: An Agile Toolkit, Mary Poppendieck, Tom Poppendieck, Addison Wesley, 2003
- [5] Agile and Lean Program Management: Scaling Collaboration Across the Organization, Johanna Rothman, Practical Ink, 2016
- [6] User Stories Applied: For Agile Software Development, Mike Cohn, Addison-Wesley, 2004
- [7] Agile Software Requirements: Lean Requirements Practices for Teams, Programs and the Enterprise, Dean Leffingwell and Don Widrig, Pearson Addison-Wesley Professional, 2010
- [8] Test-Driven Development By Example, Kent Beck, Addison Wesley, 2002
- [9] Lean-Agile Acceptance Test-Driven Development: Better Software Through Collaboration, Ken Pugh, Pearson, 2010
- [10] Lean Mindset, Mary Poppendieck and Tom Poppendieck, Addison-Wesley, 2014
- [11] Successfully transforming to Lean by changing the mindset in a global product development team, U. Samatha, V. S. Mani, IEEE 10th International Conference on Global Software Engineering, 2015